

CHAPTER 1

INTRODUCTION

1.1 LARGE LANGUAGE MODELS IN CODE GENERATION

The emergence of large language models (LLMs) represents a major milestone in artificial intelligence and software engineering. These models, primarily based on transformer architectures, are trained on massive corpora consisting of source code, technical documentation and natural language text. As a result, they can generate syntactically valid and semantically coherent code across multiple programming languages and domains.

Code generation the automatic synthesis of executable programs from natural language or partial code has long been a goal in software engineering. Earlier approaches, including rule based systems, template driven methods and sequence to sequence models, were limited in scope, struggled with ambiguity and lacked cross domain generalization. LLMs overcome these limitations by leveraging large scale pretraining, enabling transferability across tasks, languages and abstraction levels.

Modern LLM based systems such as GitHub Copilot, Codex, AlphaCode, and Code Llama demonstrate strong capabilities in tasks like function completion, test generation, code translation. Their widespread adoption in industry highlights both their practical utility and the importance of understanding their limitations. Despite these advances, LLM based code generation is not fully reliable. The key issue is hallucination, i.e., the generation of code that appears correct but is logically flawed, non-functional or based on non-existent APIs [2]. These errors can have serious consequences in real world software systems. Therefore, understanding and detecting hallucinations in LLM generated code remains a critical area for research.

1.2 HALLUCINATIONS IN CODE GENERATIONS

Despite the advancements in Large Language Models (LLMs) for code generation, ensuring the correctness and reliability of generated code remains a significant challenge. One of the major issues is hallucination, where the model produces syntactically correct but semantically incorrect code, including invalid API usage, logical inconsistencies and runtime failures.

These hallucinations are difficult to detect early, as generated code often appears plausible and may pass limited tests but fails in broader or edge-case scenarios. This makes automated detection essential, since superficial correctness can hide deeper semantic and execution-level flaws.

Existing approaches such as prompt engineering, repeated sampling, supervised fine-tuning and reinforcement learning from human feedback (RLHF)[1] provide partial improvements. However, these methods do not guarantee program correctness and often fail to address deeper structural and execution level errors.

Another key challenge arises in real world environments where data required for improving model performance is distributed across multiple clients. Due to privacy and data ownership concerns, this data cannot be centrally collected for training. Traditional centralized learning approaches are therefore not suitable for such scenarios.

Furthermore, standard federated learning methods, such as Federated Averaging (FedAvg), aggregate model updates based only on dataset size and do not consider the variation in error patterns across clients. This limits the model's ability to effectively learn from diverse hallucination distributions.

These challenges highlight the need for a framework that ensures reliable code generation while enabling decentralized, privacy preserving learning with improved aggregation strategies.

1.3 FEDERATED LEARNING

The limitations of existing code generation approaches highlight the need for a more robust and scalable solution that ensures both correctness and adaptability. While Large Language Models (LLMs) are capable of generating code efficiently, their outputs often lack reliability due to hallucinations and insufficient validation mechanisms.

At the same time, real world data required to improve model performance is distributed across multiple clients and cannot be centrally aggregated due to privacy and data ownership constraints. This creates a need for decentralized learning approaches that can utilize distributed data without compromising privacy.

To address these challenges, a system is required that not only detects and corrects hallucinations but also enables collaborative learning across multiple clients. Furthermore, traditional federated learning approaches like FedAvg [3] are not sufficient, as they do not consider the variation in error patterns across different clients[9][13]. Hence, there is a need for an improved aggregation strategy that can effectively utilize error information during model updates.

The proposed system addresses these requirements by combining hallucination-aware processing with a federated learning framework and introducing an error aware aggregation method to enhance overall model performance

1.4 PARAMETER EFFICIENT FINE TUNING

The scale of modern large language models ranging from millions to hundreds of billions of parameters makes full fine tuning computationally expensive and often impractical without large GPU infrastructure. It requires

updating all parameters, storing gradients and maintaining optimizer states, leading to significant memory and compute overhead. Parameter efficient fine tuning (PEFT) [5] addresses this by adapting a mostly frozen pre trained model using a small set of trainable parameters, significantly reducing resource requirements.

Among PEFT methods, Low Rank Adaptation (LoRA) [11], represents weight updates as the product of two low rank matrices, thereby reducing trainable parameters. Other approaches include prompt tuning, which inject trainable vectors into inputs or hidden states and adapter modules, which add small bottleneck layers within transformer blocks. These methods trade slight reductions in flexibility for major gains in efficiency.

For code generating LLMs, PEFT is particularly valuable in reducing hallucinations as it enables targeted fine tuning on curated datasets without overwriting general knowledge. This helps improve API correctness and generation reliability while preserving overall capability. When combined with federated learning, PEFT further reduces communication costs, since only lightweight adapter parameters need to be shared instead of full model updates.

1.5 PROBLEM STATEMENT

Large Language Models (LLMs) used for code generation often produce hallucinated outputs, resulting in syntactically correct but semantically incorrect code. These inaccuracies reduce the reliability of such models in real world software development scenarios. While existing approaches attempt to improve performance through fine tuning and prompt engineering, they do not effectively leverage distributed data available across multiple clients.

In practical environments, data required for improving model performance is decentralized and cannot be shared due to privacy and ownership constraints.

Although federated learning provides a solution for decentralized training, traditional aggregation methods such as Federated Averaging (FedAvg) rely only on dataset size and fail to consider variations in error patterns across clients.

This limits the model's ability to learn effectively from diverse hallucination distributions. Therefore, there is a need for a privacy preserving framework that not only enables collaborative learning across distributed clients but also incorporates error aware aggregation strategies to improve the reliability and correctness of LLM based code generation.

1.6 OBJECTIVE

The primary objective of this project is to design, implement and evaluate a federated learning based framework that improves the reliability of LLM based code generation through privacy preserving, error aware fine tuning. The specific goals of the project are:

1. Develop a federated learning framework using LoRA based PEFT to enable collaborative LLM fine tuning across decentralized clients on private codebases while reducing computational and communication overhead.
2. Design a hallucination-aware pipeline using static and dynamic program analysis to detect and classify syntactic, semantic, API related and logical errors in generated code through a structured taxonomy.
3. Propose Federated Error Average, a novel error aware aggregation strategy that incorporates client side hallucination error distributions to enhance global model robustness over heterogeneous, non-IID data.
4. Evaluate the proposed framework on benchmark datasets using hallucination metrics to validate improvements in functional correctness and reductions in hallucination frequency and severity.

1.7 ORGANIZATION OF THE REPORT

- **Chapter 1** introduces the role of LLM's in code generation and the issues caused by hallucinations. It then discusses the challenges of handling diverse error patterns in distributed settings and the limitations of standard Federated Averaging. The chapter presents parameter efficient fine tuning using LoRA as a practical approach and concludes with the problem statement and the project objectives, including hallucination detection, federated LoRA fine tuning, the Federated Error Average strategy and evaluation metrics.
- **Chapter 2** provides a literature survey of works on LLM based code generation, hallucination detection and mitigation, efficient fine tuning and federated learning. It reviews the approaches and identifies key gaps such as limited integration of execution based detection, lack of error aware aggregation and challenges in decentralized settings that motivate the proposed framework.
- **Chapter 3** presents the architecture of the proposed hallucination-aware federated framework. It describes the client side pipeline for code generation, static and dynamic analysis and dataset creation, along with the server side aggregation using LoRA adapters. The chapter also outlines the Federated Error Average strategy for error aware aggregation.
- **Chapter 4** details the implementation of the system, including datasets, tools and model configuration. It covers the integration of detection modules, dataset construction, LoRA fine tuning setup and federated aggregation using both Federated Average and Federated Error Average along with the evaluation setup for generalization.
- **Chapter 5** presents the results and evaluation across HumanEval, MBPP and DS1000. It compares baseline, FedAvg and Federated Error

Average approaches using hallucination counts, pass rates and CodeBLEU scores, highlighting improvements and remaining limitations.

- **Chapter 6** concludes the work by summarizing the contributions in hallucination-aware training and federated aggregation. It also outlines future directions such as scaling to larger models, improving repair mechanisms and extending the framework to more complex datasets and multi round training.

CHAPTER 2

LITERATURE SURVEY

This chapter presents a comprehensive review of existing research relevant to federated learning for LLM code generation with hallucination-aware fine-tuning. The survey is organized into four sections namely Code Generation and Reliability of LLM Outputs, Hallucination Detection and Mitigation Techniques, Efficient Fine-Tuning of Code Generation Models and Federated Learning and Aggregation Strategies, followed by a Summary of Literature and Research Gap.

2.1 CODE GENERATION AND RELIABILITY OF LLM OUTPUTS

Prompt engineering and evaluation of LLM-generated code focus on understanding how input design, self verification and testing strategies influence code quality and reliability. This area examines the strengths and limitations of prompting .

The Impact of Prompt Programming on Function Level Code Generation

Khojah et al. [2] analyzed the impact of prompt programming on function level code generation using the CodePromptEval benchmark of 7,072 prompts across five techniques (few-shot, chain-of-thought, persona, function signature and package context) on 221 real world Python functions. Through a controlled empirical study varying prompt structures to measure syntactic correctness, functional accuracy and complexity, the researchers aimed to isolate prompt engineering effects from underlying model capacity. While structured prompts reduced minor surface level errors, the study concludes that prompt engineering remains highly sensitive to task type, lacks robustness across diverse tasks and does not eliminate deeper logical or semantic errors or address hallucination systematically findings that directly motivate combining prompts with

automated detection and federated adaptation in our FedError Average pipeline.

This limitation reveals that prompt tuning alone cannot guarantee code correctness without post-generation validation. Therefore, our approach shifts focus from improving input prompts to correcting output hallucinations through multi-client aggregation. By leveraging error distribution across heterogeneous clients, FedErrorAvg compensates precisely for the inconsistencies that prompt engineering fails to resolve.

Fight Fire With Fire: Trustworthiness of ChatGPT on Code Tasks

Yu et al. [10] evaluated ChatGPT's self verification capability across code generation, completion and program repair using three prompting strategies (direct question, guiding question and test report prompting), measuring accuracy, precision, recall and F1-score while specifically analyzing self contradictory hallucinations. The findings reveal that model performance varies significantly across datasets and task complexity, with ChatGPT frequently overestimating the correctness of its own outputs leading to low recall in direct self verification and when test reports improve recall, precision is often reduced. The study highlights the presence of hallucinations and inconsistencies but does not propose a concrete mitigation framework, underscoring the need for an error aware federated fine-tuning pipeline.

No Need to Lift a Finger Anymore? Assessing the Quality of Code Generation by ChatGPT

Liu et al. [4] conducted a large-scale assessment of LLM-generated code quality covering 728 LeetCode algorithm problems and 54 Common Weakness Enumeration (CWE) security scenarios across five programming languages, using the LeetCode judge system for functional correctness and CodeQL for static security analysis while exploring multi round dialog based fixing with

iterative feedback. The study found that while LLM outputs are often syntactically correct, they frequently fail functionally due to logical errors and incorrect assumptions, with ChatGPT performing significantly better on problems published before 2021 indicating reliance on memorized patterns rather than genuine reasoning and the dialog based fixing process showing limited effectiveness for logically complex problems, highlighting reasoning limitations without providing mechanisms for correcting hallucinated outputs or pooling evidence across distributed clients.

LLM-Based Test Driven Interactive Code Generation

Fakhoury et al. [1] conducted a user study and empirical evaluation of LLM-based test driven interactive code generation, examining how developers interact with LLMs when test cases are used as specifications. The study found that while test driven prompting improves output correctness in some contexts, users often struggle to formulate effective test cases and the iterative process does not guarantee convergence to correct solutions. This further reinforces the need for automated, execution grounded validation rather than relying solely on user-driven testing or prompt refinement.

2.2 HALLUCINATION DETECTION AND MITIGATION TECHNIQUES

Hallucination detection in LLM generated code focus on identifying and mitigating syntactic, semantic and logical errors produced during code generation. This area explores hybrid analysis techniques, execution-based verification and knowledge guided correction methods to improve code correctness.

Advancing LLM Generated Code Reliability: A Hybrid Approach for Hallucination Detection

Yang et al. [8] proposed SDHD (Static-Dynamic Hybrid Detection), a framework integrating static analysis (via Abstract Syntax Trees) and dynamic execution (via runtime testing) to detect both structural and functional errors, categorizing hallucinations into eight distinct types while achieving superior precision, recall and F1-score across multiple benchmarks. The primary limitation is that dynamic detection depends heavily on test coverage quality; incomplete test suites may miss certain logical failures and the framework is currently validated only on Python, limiting cross-language generalizability, with scalability in distributed environments remaining a challenge that our project addresses by feeding detection outcomes into decentralized LoRA fine-tuning via the FedError Average mechanism.

CodeHalu: Investigating Code Hallucinations via Execution-Based Verification.

Tian et al. [14] proposed CodeHalu, an execution-based hallucination verification framework foundational to our taxonomy, where the framework executes LLM generated code against test cases under time and memory constraints while recording execution failures, unexpected outputs and infinite loop behaviors, using a two stage heuristic induction process to classify hallucinations into four main types (Mapping, Naming, Resource, Logic) with eight subcategories, supported by the CodeHaluEval benchmark of 8,883 samples from 699 tasks. The study focuses exclusively on Python, leaving hallucination behaviors in other programming languages unevaluated. This taxonomy directly informs our project's client side detection module, though we acknowledge the need for cross language validation.

THINK: Tackling API Hallucinations in LLMs via Injecting Knowledge

Liu et al. [15] proposed THINK, a two-phase API hallucination mitigation framework that constructs an API knowledge database using Javadoc and validated code examples, retrieving task-relevant APIs via task decomposition and similarity search and classifying errors into seven API error types for correction. The framework is limited to API-level errors and does not address broader logical or structural hallucinations, while the ReAct-based repair depends on LLM generation leading to non-deterministic behavior and it does not exploit cross-client diversity of failure modes during federated updates.

EvoAPR: Enhancing LLMs for Automatic Program Repair with Genetic Algorithm and Dynamic LoRA.

Zhang et al. [11] proposed EvoAPR, combining genetic algorithms with LLM-based program repair, where genetic evolution operates under a tri-objective fitness function covering syntax compliance, bug hunk masking rate and repair location masking rate, while Dynamic LoRA swaps non-overlapping rank-r adapters per bug index for on demand, project specific parameter retrieval. The system is computationally expensive and may not scale efficiently for large datasets or real time applications and it does not target federated aggregation weighted by hallucination statistics distinguishing it from our proposed framework.

2.3 EFFICIENT FINE-TUNING OF CODE GENERATION MODELS

Efficient fine-tuning of code generation models focuses on adapting large language models to programming tasks using parameter-efficient techniques such as LoRA, while minimizing computational cost and data requirements. This area explores methods to improve code quality and domain adaptability.

Towards Efficient Fine-Tuning of Language Models With Organizational Data

Nashaat and Miller [5] proposed CodeMentor, a framework for few-shot learning utilizing heuristic rules and weak supervision, employing Low-Rank Adaptation (LoRA) for parameter efficient fine tuning and integrating Reinforcement Learning from Human Feedback (RLHF) to incorporate domain expertise from organizational data. The approach is designed for centralized environments and does not consider distributed data scenarios, with the RLHF bottleneck during scaling and domain bias limiting generalization, while also failing to address hallucination issues in generated code, limiting its applicability to privacy sensitive federated deployments.

EvoAPR: Dynamic LoRA for Program Repair

As described in Section 2.2, Zhang et al. [11] introduced Dynamic LoRA swapping for project specific parameter retrieval in program repair contexts. While this demonstrates the viability of LoRA for code specific tasks, the approach is computationally expensive and does not target federated aggregation weighted by hallucination statistics, distinguishing it from our proposed framework.

2.4 FEDERATED LEARNING AND AGGREGATION STRATEGIES

Federated learning and aggregation strategies focus on enabling decentralized model training across multiple clients while preserving data privacy by sharing only model updates instead of raw data. Within this paradigm, aggregation methods play a crucial role in combining client-side updates into a global model, addressing challenges such as data heterogeneity.

A Federated Adaptive LLM Fine-Tuning Framework for Software Development

Chen et al. [3] proposed a federated adaptive fine-tuning framework for large language models in software development, enabling decentralized training across multiple clients while preserving data privacy by sharing only model adapter weights, using FedAvg style aggregation with an L2-norm constraint to combine adapters based on local data size to manage gradient divergence. Although the framework improves adaptability across distributed datasets, it does not incorporate error aware learning or hallucination-specific optimization, with aggregation still based on standard federated averaging limiting effectiveness when clients exhibit diverse defect distributions, exactly the scenario targeted in the proposed work.

FedALoRA: Adaptive Local LoRA Aggregation for Personalized Federated Learning

Yi et al. [9] proposed FedALoRA, which combines low-rank adaptation with federated learning using an Adaptive LoRA Aggregation (ALoRA) mechanism to merge global and local parameters via locally learned weights with element-wise fusion focused on higher transformer layers, selectively preserving domain expertise while capturing shared knowledge without increasing communication costs. The framework specifically targets the non-IID problem common in real-world federated deployments, where cross-source or cross-domain data distributions differ significantly across clients, and addresses this by initializing the local model before each iterative training round so that clients can simultaneously learn general knowledge and reinforce domain-specific understanding. The approach focuses on general NLP tasks and does not consider hallucination optimization in code generation, specifically failing to exploit error aware aggregation which motivates the present work.

LEGEND: Adaptive Parameter-Efficient Federated Fine-Tuning on Heterogeneous Devices.

Liu et al. [13] presented LEGEND, a LoRA-based federated fine-tuning framework that adapts LoRA depth and rank placement for resource constrained and heterogeneous devices, reporting substantial speedups and communication savings on a large device testbed. While highly relevant to efficient cross client training, LEGEND optimizes compute and bandwidth rather than incorporating cross client distributions of hallucination-related faults into the aggregation rule the distinguishing feature of the proposed framework.

Flexible and Secure Code Deployment in Federated Learning using Large Language Models

Seo et al. [12] studied how LLMs can tailor deployment artifacts across heterogeneous federated clients, introducing prompt-engineering techniques to improve malicious code detection before execution to strengthen trust in automated code distribution pipelines where human access to each node is limited. The emphasis is on security screening rather than on reducing semantic hallucinations in generated application code or on weighting federated updates by categorized coding errors, leaving room for complementary hallucination-aware training as proposed in this project.

2.5 SUMMARY OF LITERATURE

The literature survey reveals several key insights that directly inform the design of the proposed federated hallucination-aware code generation framework.

Key Finding 1: LLM Code outputs remain unreliable

Prompting alone and single-node evaluation studies [5][6][8] do not remove deep semantic or functional failures in LLM-generated code. While structured prompts improve surface level quality, logical and execution level hallucinations persist, motivating execution-grounded detection [1] and structured repair pipelines [9][10][11]. This establishes the need for the client side hallucination detection module in the proposed framework.

Key Finding 2: Federated adapter training is feasible but lacks error awareness

Federated adapter training for LLMs is demonstrated to be viable [3][7], but standard aggregation strategies ignore variation in defect types across clients. This is particularly problematic in code generation, where failures are structured, enumerable and distributed unevenly across organizations. The proposed FedError Average mechanism directly addresses this gap by weighting client contributions based on their hallucination error distributions.

Key Finding 3: Heterogeneous LoRA configurations are actively researched

Recent work [13][14] has made progress in handling diverse device capabilities and varying LoRA configurations in federated environments. These advances improve how model updates are aligned across different clients. However, they do not incorporate hallucination error patterns. The proposed framework builds on these foundations while introducing a hallucination weighted aggregation mechanism that prior work lacks.

Key Finding 4: Execution based hallucination classification provides rich structure

The CodeHalu taxonomy [14] and the hybrid static-dynamic detection

approach of Yang et al. [8] demonstrate that code hallucinations can be systematically classified into structured categories. This validates the design of the proposed client-side detection module, which uses AST analysis, library API validation and dynamic execution testing to generate structured error labels that drive hallucination-aware dataset construction and federated aggregation weighting.

Research Gap: The existing literature lacks a unified framework that integrates multiple critical aspects of hallucination-aware code generation. Specifically, prior works do not simultaneously combine (i) client side static and dynamic hallucination analysis tailored for code LLMs (ii) privacy preserving LoRA based federated learning without centralizing raw code and (iii) aggregation strategies that prioritize clients with richer hallucination error patterns. While these components have been individually explored such as hybrid static dynamic detection approaches [8] or federated LoRA fine-tuning techniques [3][13] there remains a clear gap in integrating them into a single cohesive system. Moreover, existing solutions often fail to address challenges such as heterogeneous defect distributions across clients, data privacy constraints and end to end interpretability.

To bridge this gap, this project proposes a unified Fed-Error Average pipeline that tightly integrates client-side hallucination detection, information gathering and federated LoRA fine-tuning, thereby enabling a privacy preserving, interpretable and robust framework for improving the reliability of LLM-generated code.

CHAPTER 3

HALLUCINATION-AWARE FEDERATED FRAMEWORK

3.1 SYSTEM OVERVIEW

The proposed system is organized into two tiers: the client tier and the server tier. These tiers interact through parameter exchange mechanism, where only LoRA adapter weights are shared, never the raw data.

At the client tier, each participating client operates as a fully independent code generation and fine-tuning unit. A client receives a benchmark dataset or dataset partition, generates code using the frozen base LLM then subjects the generated code to a hallucination detection pipeline and constructs a local training dataset from the outcomes. The client then performs supervised fine-tuning (SFT) using LoRA on this locally assembled dataset, producing a set of client specific adapter weights. These weights encode the learning the model derived from that client's error patterns and corrected programs.

The server tier consists of a central aggregation server whose sole responsibility is to collect LoRA adapters from all clients at the end of each communication round, perform weighted averaging to produce a global adapter and redistribute that global adapter back to all clients. The server performs no code generation or evaluation; it is purely a coordinator. It is important to note that it never has access to the raw datasets, generated code or problem statements from any client. The client-side hallucination detection and dataset construction pipeline is illustrated in Figure 3.1, while the LoRA fine-tuning and federated aggregation process is illustrated in Figure 3.2.

3.2 OVERALL ARCHITECTURE

3.2.1 Client-Side Hallucination Detection and Dataset Construction

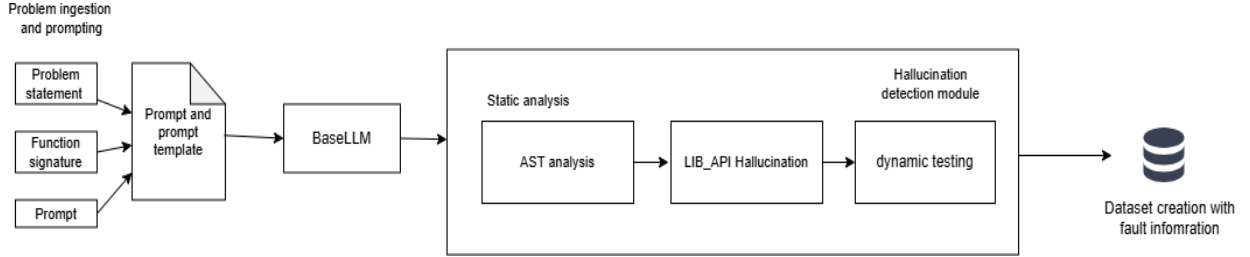


Figure: 3.1 – Client-side hallucination detection and dataset construction framework

The first phase is executed entirely within each client and encompasses all activities from code generation through dataset preparation, as illustrated in Figure 3.1. It begins with problem ingestion, where benchmark datasets (HumanEval, MBPP and DS- 1000) are loaded and each client is assigned a distinct dataset or partition to simulate heterogeneous data distributions in a real federated environment. For each task, a structured prompt is constructed by combining the problem description and the function signature. This prompt is then fed to the frozen Qwen2.5-Coder-3B [16] model using single shot prompting, producing an initial code solution.

Once code is generated, it enters the hallucination detection pipeline. Static analysis is performed first using Python's AST module, which parses the generated code and inspects its structural representation. Following static analysis, a library and API validation step checks whether the functions, methods and attributes called in the code correctly correspond to the actual interfaces provided by the libraries available in the execution environment.

After static analysis, the code undergoes dynamic analysis by executing it against the test cases provided by the benchmark dataset. Execution takes place in a controlled environment with timeout protection to prevent infinite loops from stalling the pipeline. Any runtime exceptions including `AssertionError`, `NameError`, `TypeError`, `ValueError`, `IndexError` and others are captured along with their traceback information, including error type, error message and the line number where the failure occurred. Failures in assertion checks, which indicate semantic or logical errors, are also recorded alongside the expected and actual outputs for each test case.

These fault records, together with the original prompt, the generated code and the canonical solution from the dataset, form the raw material for dataset construction.

Dataset construction organizes the above information into a structured format with columns including the dataset name, task identifier, prompt, status, AST analysis information, dynamic analysis information, library analysis information, the generated code, the target code (canonical solution), error sources, error types and error lines. The prompt and canonical solution columns are the primary training signal used in the subsequent LoRA fine-tuning stage. The error type and frequency information is recorded separately to be reported to the server for use in the FedError Average aggregation.

3.2.2 LoRA Fine-Tuning and Federated Aggregation

The second phase (Figure 3.2), begins once the client has assembled its dataset from Phase 1. The base Qwen2.5-Coder-3B model is loaded and base model is frozen and LoRA adapter modules are injected into all seven projection matrices of each transformer layer. This comprehensive injection ensures that the adapter can influence both the attention and feed forward computations across all 36 transformer layers of the model (QwenCoder).

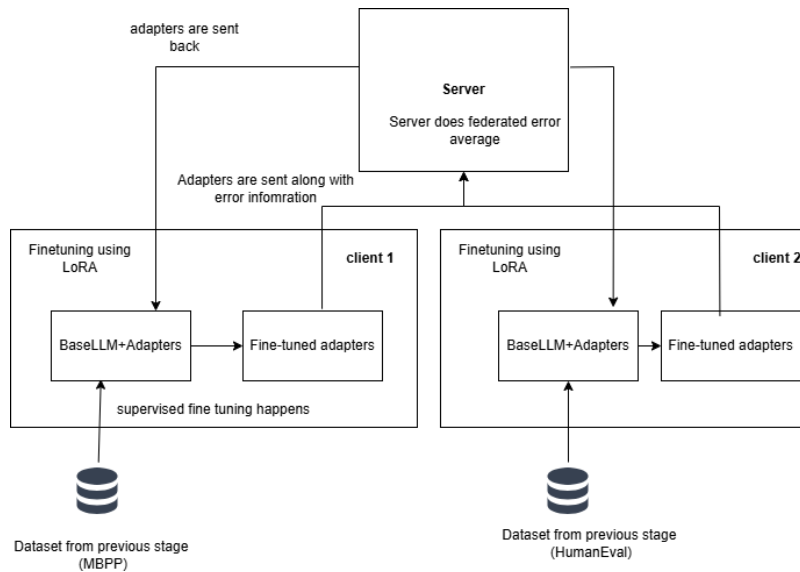


Figure: 3.2 LoRA Fine tuning and federated aggregation

The LoRA rank is set to 16 and the scaling factor alpha to 32, yielding an effective scaling multiplier of 2. With these setting we get 504 adapter in total, corresponding to 36 layers. The total number of trainable LoRA parameters across the model is approximately 29.9 million, which is a small fraction of the full 3.09 billion parameters. All original model weights remain strictly frozen throughout training and only the LoRA adapter matrices are updated.

Training is conducted using supervised fine-tuning (SFT). After training completes, the LoRA adapter weights are saved as a “.pt” checkpoint file and prepared for transmission to the central server. No model weights, no raw data and no code examples leave the client at this stage; only the compact adapter checkpoint is sent.

At the server, the FedError Average aggregation algorithm is applied. First, the server collects the adapter checkpoints and the error frequency reports from all participating clients. For each error type e observed across all clients, the

total error frequency F_e is computed by summing the counts of that error type across all clients. A normalized weight is then assigned to each error type based on its global frequency, reflecting how prevalent that category of error is across the federated system. For each client k , an error score E_k is computed by multiplying the count of each error type on that client by the corresponding global weight and summing across all error types. This score quantifies how rich or diverse the client's error distribution is relative to the overall population.

Each client's aggregation weight α_k is obtained by normalizing its error score E_k against the sum of all clients' error scores. These weights replace the sample size based weights used in standard FedAvg. The global adapter at round $t+1$ is then computed as the weighted average of all clients adapter weights at round t , with each client weighted by its α_k . The resulting global adapter is redistributed to all clients, who load it alongside the frozen base model to obtain an improved code generation system. This completes one federated communication round and the process can be repeated for subsequent rounds.

3.3 MODULES BREAKDOWN

The proposed system is composed of six modules:

1. Code Generation Module.
2. Hallucination Detection Module.
3. Dataset Creation Module.
4. LoRA Fine-Tuning Module.
5. Federated Aggregation Module.
6. Verification Module.

3.3.1 Code Generation Module

The Code Generation Module produces initial program solutions from benchmark datasets using a frozen large language model. As the first stage of the

pipeline, this module allows each client to independently generate candidate code without any adaptation, thereby simulating real world federated environments.

The module first loads three benchmark datasets HumanEval, MBPP and DS-1000 from the Hugging Face datasets library defined as “D” in algorithm 3.1. Each dataset is assigned to a different client to reflect heterogeneous data distributions. For every task, a structured prompt is constructed by combining the problem description with the function signature. Dataset specific preprocessing is applied as per design decisions.

The resulting prompt is fed into the frozen Qwen2.5-Coder-3B[16] model using greedy decoding (`do_sample=False`, `temperature=1.0`), ensuring deterministic and reproducible outputs across clients. The generated code is stored along with the task ID and the corresponding prompt.

Algorithm 3.1 Code Generation using Frozen LLM

Input: Benchmark dataset D

Output: List of (task_id, prompt, generated_code)

```

1: Load dataset  $D$  from Hugging Face
2: for each task  $t \in D$  do
3:   Construct structured prompt: using description and function signature
4:   If MBPP, extract first def line as signature
5:   If DS-1000, truncate prompt after "A:"
6:   Generate code using frozen LLM with greedy decoding
7:   generated_code  $\leftarrow$  LLM(prompt)
8:   Store (task_id, prompt, generated_code)
9: end for
10: return list of generated tuples

```

Design Decisions:

- Greedy decoding (`do_sample=False`, `temperature=1.0`) is used to ensure reproducibility across clients.
- Each client is assigned a distinct dataset to simulate heterogeneous data distributions in a real federated environment.
- For MBPP, the function signature from the reference code (first “def” line)

is appended to the prompt to align with benchmark test cases.

- For DS-1000, the verbose prompt is preprocessed by splitting on 'A:' and retaining only the question portion as `prompt_v2`.

3.3.2 Hallucination Detection Module

The hallucination detection module is the analytical core of the client-side pipeline, accepting generated code and producing a structured fault record through two sequential stages: static analysis and dynamic analysis.

The AST module first inspects code without execution by parsing Python source into a tree. If parsing fails due to syntax or indentation errors, it records the error and line number and stops. If parsing succeeds, it traverses the tree to verify control-flow correctness (return only in functions, break or continue only in loops) and performs flow aware name analysis to detect undefined variables across conditionals, loops, try-except blocks and nested scopes. It detects syntax errors, indentation errors, structural violations and undefined variables purely from code structure, even in rarely executed paths. Any reported error skips later stages.

The LIB_API module then performs static checks on external libraries and APIs without running code, validating imports, attribute access and function calls using `importlib` and `inspect`. It reports missing modules, invalid attributes and invalid argument signatures, distinguishing between missing libraries and actual API misuse.

Finally, the dynamic analysis module executes the code against benchmark test cases in an isolated environment with a timeout. If all tests pass, the code is marked passed. Otherwise, it records failures including exception type, message, line number, failing input, expected output and actual output. This module detects runtime errors, but only on paths executed by tests, which is why the static AST and LIB_API modules complement it.

Algorithm 3.2 Hallucination Detection Pipeline

Input: generated_code, benchmark_test_cases
Output: Unified fault record

- 1: **Stage 1: AST Analysis (Static)**
- 2: Parse code using AST module
- 3: **if** SyntaxError or IndentationError **then**
- 4: Record error details and terminate pipeline
- 5: **end if**
- 6: Traverse AST and validate control-flow rules
- 7: Check **return** inside functions
- 8: Check **break/continue** inside loops
- 9: Perform flow-aware name analysis across all paths
- 10: **if** errors detected **then**
- 11: Build fault record and skip remaining stages
- 12: **end if**
- 13: **Stage 2: Dynamic Analysis**
- 14: Execute code with test cases in isolated environment (timeout = 10s)
- 15: **if** all tests pass **then**
- 16: Mark status = PASSED
- 17: **else**
- 18: Record runtime errors: type, message, line number
- 19: Record expected vs actual outputs
- 20: **end if**
- 21: **Stage 3: LIB/API Validation**
- 22: **if** dynamic analysis failed **then**
- 23: Validate imports using `importlib`
- 24: Validate attributes and function signatures using `inspect`
- 25: Record module_not_found, attribute, type, name errors
- 26: **end if**
- 27: **return** unified fault record

Design Decisions:

- The pipeline is sequential: AST $\rightarrow$ Dynamic $\rightarrow$ LIB_API, with early exit on AST failure to avoid executing syntactically invalid code.
- Static analysis catches errors on unexecuted paths (e.g., rarely triggered branches), which dynamic testing cannot detect.
- LIB_API uses `importlib` and `inspect` to distinguish genuine API hallucinations from environment level missing packages, preventing false positives.
- Threading with a 10 second timeout is used in dynamic analysis to prevent infinite loops from stalling the pipeline.

3.3.3 Dataset Creation

The dataset construction module transforms raw hallucination detection outputs into a structured training dataset for LoRA fine-tuning. For each task, the module assembles a record containing the dataset name, task identifier, prompt, hallucination status, AST analysis, dynamic analysis, library analysis, generated code, canonical target code, error sources, error types, error lines and the benchmark's canonical solution.

Although Automatic Program Repair (APR) was initially integrated to correct hallucinated code using detected fault information, it was observed that the model frequently failed to generate valid patches even with precise error localization. To ensure training reliability, canonical solutions are consistently used as ground-truth targets for failed tasks, while error frequency information is maintained separately and utilized only during federated aggregation for computing error-aware weights.

An automatic program repair (APR) module was initially intended to correct hallucinated code using detected error information, but it often failed to produce valid fixes due to limitations of the Qwen coder model. Therefore, the system does not rely on APR outputs as training targets. Instead, programs that passed all hallucination checks are stored with the generated code as the target, while programs that failed detection are stored with the canonical solution from the benchmark as the target. The prompt and canonical solution columns serve as input–output pairs for supervised fine-tuning, ensuring that fine-tuning always uses a correct reference target even when the model's own generation was hallucinated.

The main aim is hallucination reduction in a federated setting rather than through detection and repair. Thus, the hallucination framework is used to assess code correctness via hallucination count.

Algorithm 3.3 Fault Information Processing for Dataset Construction

Input: Fault records for all tasks on a client

Output: Dataset CSV, error frequency report

```

1: for each task  $t$  do
2:   Retrieve fault record:
3:     status, error_type, error_line, generated_code, canonical_solution
4:   if status == PASSED then
5:     Set target_code = generated_code
6:   else
7:     Set target_code = canonical_solution
8:   end if
9:   Assemble structured record:
10:    (prompt, target_code, error_type, error_line, ...)
11:   Append record to dataset CSV
12: end for
13: Compute error frequency report:
14:   Count occurrences of each error_type
15: return dataset CSV and error frequency report

```

Design Decisions:

- The Automatic Program Repair (APR) was initially planned to correct hallucinated code [11].
- Although Automatic Program Repair (APR) was integrated into the pipeline to correct hallucinated code using detected fault information, it was found to be unreliable in practice the Qwen model frequently failed to produce valid patches even when provided with precise error types and line-level fault locations.
- Canonical solutions are used as training targets for failed tasks, ensuring fine-tuning always uses a correct reference regardless of model hallucination.
- The error frequency report is kept separate from training data and is transmitted to the server exclusively for FedError Average weight computation.

3.3.4 LoRA Fine-Tuning Module

The LoRA fine-tuning module performs parameter-efficient supervised fine-tuning of the frozen base model using the hallucination-aware dataset from the dataset construction module. Low-Rank Adaptation (LoRA) decomposes weight updates into low-rank matrix products, drastically reducing trainable parameters while preserving expressive capacity.

LoRA adapters are injected into all seven projection matrices per layer across all 36 transformer layers of the Qwen2.5-Coder-3B-Instruct model, covering both attention projections (q_proj, k_proj, v_proj, o_proj) and MLP projections (gate_proj, up_proj, down_proj), resulting in 504 adapter insertion points. The base model weights remain fully frozen throughout training and only the LoRA adapter matrices are updated via gradient descent.

Algorithm 3.4 LoRA Fine-Tuning for Hallucination-Aware Learning

Input: Dataset D , frozen base model M

Output: LoRA adapter checkpoint W

- 1: Inject LoRA adapters into model:
 - 2: For each layer, add adapters to q_proj, k_proj, v_proj, o_proj
 - 3: Add adapters to gate_proj, up_proj, down_proj
 - 4: Freeze all original model weights
 - 5: Format dataset as prompt completion pairs
 - 6: Train model using supervised fine-tuning (SFTTrainer)
 - 7: Use completion-only loss
 - 8: Apply gradient accumulation
 - 9: Save trained LoRA adapter weights:

$$W = \{A_i, B_i\}_{i=1}^{504}$$
 - 10: **return** adapter checkpoint
-

The dataset is formatted into prompt-completion pairs and training uses a completion-only loss with an effective batch size of 8 achieved through gradient accumulation (per-device batch size of 1, accumulation steps of 8) for 3 epochs. The LoRA configuration uses rank $r=16$ and $\alpha=32$, producing a scaling factor of 2 that balances expressiveness and efficiency. After training, only the

learned LoRA adapter weights are saved as a .pt checkpoint file, ensuring lightweight updates that preserve data privacy and reduce communication overhead in the federated setting.

Design Decisions:

- The LoRA rank $r=16$ and $\alpha=32$ yield a scaling multiplier of 2 (α/r), balancing expressiveness and parameter efficiency.
- All 7 projection matrices (attention + MLP) are targeted to allow the adapter to influence both attention computation and feed-forward representations.
- Gradient accumulation (steps=4) is used to achieve an effective batch size of 8 while staying within GPU memory limits.
- Only adapter weights are saved and transmitted raw data, model weights and code examples never leave the client.

3.3.5 Federated Aggregation Module

The federated aggregation module operates on the central server and combines LoRA adapters received from all clients into a single global adapter using the proposed FedError Average algorithm. This method extends standard Federated Averaging (FedAvg) by incorporating hallucination error frequency information alongside dataset size, making the aggregation process aware of the types of errors encountered by each client.

Each client sends only the adapter checkpoints along with an error frequency. The server never accesses raw datasets, generated code or prompts, thereby preserving data privacy. The aggregation process begins by computing, for each error type, the global error frequency as the sum of counts across all clients. A normalized weight for each error type is derived by dividing by the total count of all error types observed globally.

For each client, an error score is computed as the weighted sum of the client's per-type error counts using the global error type weights, reflecting not just data volume but how diverse and significant the client's error distribution is relative to the global error. The client's aggregation coefficient is obtained by normalizing against the sum of all clients' error scores. The global adapter at round $t+1$ is computed as the element-wise weighted sum over all clients of their coefficient multiplied by their local adapter weights, applied across all 504 adapter matrices. The resulting global adapter is then sent back to all clients, who merge it with the frozen base model for inference or further local fine-tuning.

The advantage over standard FedAvg is that clients with more complex or diverse hallucination patterns, such as many distinct error types versus a single type, contribute proportionally more to the global update, which is particularly important in heterogeneous federated settings where different clients encounter qualitatively different failure modes in code generation.

Algorithm 3.5 Fed-Error Average: Frequency-Weighted Federated Aggregation

Input: Client adapters $\{w_k^t\}_{k=1}^K$, error counts $\{\text{count}_{k,e}\}$

Output: Global adapter w^{t+1}

1: Compute error frequency:

$$F_e \leftarrow \sum_{k=1}^K \text{count}_{k,e}$$

2: Compute error weights:

$$\text{weight}_e \leftarrow \frac{F_e}{\sum_{e' \in \mathcal{E}} F_{e'}}$$

3: Compute client scores:

$$E_k \leftarrow \sum_{e \in \mathcal{E}} \text{count}_{k,e} \cdot \text{weight}_e$$

4: Normalize client weights:

$$\alpha_k \leftarrow \frac{E_k}{\sum_{j=1}^K E_j}$$

5: Aggregate adapters:

$$w^{t+1} \leftarrow \sum_{k=1}^K \alpha_k w_k^t$$

6: **return** w^{t+1}

Design Decisions:

- Error weighted aggregation ensures clients with more diverse or complex hallucination patterns contribute proportionally more to the global update addressing the limitations of standard FedAvg in heterogeneous (non-IID) federated settings.
- The server never accesses raw datasets, generated code, or problem statements only compact adapter checkpoints and aggregate error counts are recieved
- The weighted average is applied element-wise across all 504 adapter matrices, preserving the dimensional structure of the LoRA decomposition.
- Standard FedAvg is retained as a baseline variant (weights by dataset size n_k / N) for direct comparison.

3.3.6 Verification Module

The verification module checks whether the fine-tuned LoRA adapters improve code generation quality by measuring hallucination reduction and correctness against a baseline model. It acts as the final validation stage of the pipeline, ensuring that learned adapters genuinely enhance performance rather than overfitting or introducing new errors. The module takes as input the global LoRA adapter checkpoint which produced via FedAvg or Fed-Error-Average, the frozen Qwen2.5-Coder-3B-Instruct base model and benchmark datasets (HumanEval, MBPP and DS-1000) using either full datasets or test splits.

The adapter is loaded on top of the base model using a PEFT mechanism, ensuring that only adapter weights influence inference. For each task in the dataset, the module generates code using the same structured prompting and greedy decoding strategy used during training. The generated code is then passed through the full hallucination detection pipeline to determine correctness and identify error types.

Algorithm 3.6 Verification of Federated Aggregation

Input: Global adapter, base model M , dataset D
Output: Verification CSV, comparative metrics
 1: Load model with adapter overlay (PEFT)
 2: **for** each task $t \in D$ **do**
 3: Generate code using structured prompt (greedy decoding)
 4: Run hallucination detection pipeline:
 5: AST analysis $\rightarrow$ Dynamic execution $\rightarrow$ LIB/API validation
 6: Record pass/fail status and error details
 7: **end for**
 8: Compute aggregate metrics:
 9: Hallucination count, error counts by type, CodeBLEU
 10: Compare results with baseline (no adapter)
 11: **return** verification CSV and metrics report

The results are recorded per task, including pass/fail status and detailed error metadata in a CSV format consistent with the baseline. By comparing baseline (no adapter) and adapter runs, the module measures how many more tasks pass after fine-tuning and whether the federated adapters reduce hallucination count and improves the pass rate.

Design Decisions:

- A 75:25 train-test split is introduced to rule out overfitting the 'Train' adapter variant is fine-tuned on 75% of data and evaluated on the unseen 25%.
- Base model weights remain entirely frozen during evaluation any improvement is attributable solely to the federated LoRA adapters.
- Four adapter configurations are evaluated: FedAvg Complete, FedAvg Train, FedErrorAvg Complete, FedErrorAvg Train.
- CodeBLEU is used alongside hallucination count to capture structural and semantic similarity to reference solutions, not just functional correctness.

CHAPTER 4

IMPLEMENTATION

4.1 RUNTIME ENVIRONMENT

The Runtime Enviroments used in this work form the foundation for development, experimentation and model training, enabling efficient execution of the proposed system across different stages. These environments support code development, version control and computation. The platforms utilized in this work are described below.

4.1.1 Google Colab

Google Colab is a cloud-based Jupyter Notebook environment that allows users to write and execute Python code interactively. It provides free access to GPU and TPU resources, making it suitable for training deep learning models. During the initial stages of this work, Google Colab was used with a T4 GPU for code generation and early experimentation, enabling rapid prototyping without requiring local hardware setup.

4.1.2 Visual Studio Code and Git

Visual Studio Code (VS Code) is a lightweight but powerful source code editor developed by Microsoft. Visual Studio Code (VS Code) is a lightweight but powerful source code editor developed by Microsoft. It was used as the primary development environment for hallucination detection and analysis, as well as for results interpretation. VS Code was used in conjunction with Git and GitHub for version control and file management, ensuring systematic tracking of changes across the codebase. The implementation of federated average and federated error averaging was also carried out within this environment.

4.1.3 JupyterHub

CTSKII JupyterHub is an institutionally hosted JupyterHub platform that provides users with isolated, containerized JupyterLab environments backed by persistent storage volumes. Each user is allocated a dedicated Docker container with a private home directory, ensuring data persistence across sessions. The platform offered access to an NVIDIA A4000 GPU at no cost, making it well suited for computationally intensive tasks. Local session storage was available upon server allocation and user files were retained even after container restarts. This platform was used for fine-tuning experiments and federated averaging, leveraging the A4000 GPU for efficient model training and verification.

4.2 LIBRARIES USED

Python libraries play a central role in building the proposed system, enabling efficient implementation of data processing, code analysis, model training and evaluation within a unified environment. Each library contributes to a specific part of the pipeline, supporting both static and dynamic components as well as the learning framework. The modules used in this work are summarized in the table 4.1.

4.3 DATASET USED

The three benchmark datasets (HumanEval[17], MBPP[18] and DS-1000[19]) are used to support code generation, hallucination detection and supervised fine-tuning. These datasets cover different domains, including function level correctness, basic programming tasks and data science applications, providing a balanced setup for evaluating the proposed system. A summary of these datasets can be found in Table 4.2 .

Table 4.1 Libraries used

Library/module	Role
os	File paths and directories: locate CSVs, adapters, outputs; check that files and folders exist.
re	Regular expressions for parsing code strings, cleaning text and extracting parts of error messages (data prep and hallucination detection).
ast	Parses Python to an AST and traverses it for syntax, structure and control flow (first stage of static hallucination detection).
builtins	Built-in names for LIB_API checks so builtins count as valid and reduce false positives for undefined names.
importlib	Dynamic imports for LIB_API analysis to validate imports and that libraries or APIs exist and are used correctly.
inspect	Signatures and attributes of functions or classes for LIB_API validation of calls, arguments and methods.
threading	Background threads for dynamic tests with timeouts so long running or hanging code does not block the pipeline.
torch	Tensors, model compute, adapter merging; inference, fine-tuning and federated aggregation of LoRA weights.
transformers	Base model and tokenizer (AutoModelForCausalLM, AutoTokenizer); inference and setup for fine-tuning.
peft	LoRA config and adapter layers (LoraConfig, get_peft_model, PeftModel); PEFT and loading adapters at verification.
trl	SFTTrainer / SFTConfig for supervised fine-tuning on promptsolution pairs from the hallucination-aware dataset.
accelerate	Distributed training, mixed precision, GPU use; easier SFT across devices and environments.
safetensors	Safe binary storage/load for LoRA adapters when saving per dataset weights and merging/loading for FedAvg and verification.
seaborn	Plots for pass rates, error breakdowns and baselines.

Table 4.2 Dataset summary

Dataset	Source	Task Count	Domain	Used For
HumanEval	Hugging Face (openai/evals)	164	Functional correctness	Code generation, hallucination detection, SFT
MBPP	Hugging Face (google-research-datasets/mbpp, sanitized)	377	Basic algorithms	Code generation, hallucination detection, SFT
DS-1000	Hugging Face (xlangai/DS-1000)	1000	Data science (NumPy, Pandas, etc.)	Code generation, hallucination detection, SFT

HumanEval

HumanEval[17] is used for evaluating function level code generation, where correctness is verified through execution against predefined test cases. It focuses on ensuring that generated code is functionally correct and aligns with the expected behavior. A summary of HumanEval is provided in Table 4.3.

Table 4.3 HumanEval summary

Field	Description
task_id	Unique ID (e.g. HumanEval/0).
prompt	Natural language description of the task, including docstrings and examples; input for code generation.
canonical_solution	Reference Python implementation; target during fine-tuning and for evaluation.
test	Unit test code that calls the solution and asserts correctness; used in dynamic execution.
entry_point	Name of the function under test (e.g. has_close_elements); used to run tests against the generated function.

MBPP

MBPP[18] is used to evaluate code generation from natural language descriptions of common programming tasks. It reflects the model’s ability to handle simple, routine coding patterns and produce correct implementations. A summary of MBPP is provided in Table 4.4.

Table 4.4 MBPP summary

Field	Description
source_file	Notebook or file where the task originated.
task_id	Unique ID for the task (e.g. 602, 603).
prompt	Short natural language description of the function to implement; used as input for code generation.
code	Reference implementation of the function; used as the target during fine-tuning.
test_imports	List of import statements required to run tests (often empty).
test_list	List of assert statements used for dynamic execution.

DS-1000

DS-1000[19] is used to evaluate code generation in data science contexts, where correct usage of libraries and APIs is essential. It highlights the model’s ability to generate code that is consistent with real world library based workflows. A summary of DS-1000 is provided in Table 4.5.

Table 4.5 DS1000 summary

Field	Description
task_id	Unique ID (e.g. DS0000, DS0132).
prompt	Problem text with context and an instruction such as <code>result = ...</code> / “put solution in this variable.”
reference_code	Reference implementation; target for evaluation and fine-tuning.
metadata	Library and category (NumPy, Pandas, Matplotlib, SciPy, Sklearn, TensorFlow, PyTorch, etc.).
code_context	Execution context with <code>test_execution</code> and <code>test_string</code> helpers and input generation.
generated_code_snippet	Truncated or partial generated solution (when present).
full_code	Full generated solution; main output column for evaluation.

4.4 LLM USED

Qwen2.5-Coder [16] is the latest series of Code-Specific Qwen large language models (formerly known as CodeQwen). Significantly improvements in code generation, code reasoning and code fixing. Base on the strong Qwen2.5, we scale up the training tokens into 5.5 trillion including source code, text-code grounding, Synthetic data, etc. Qwen2.5-Coder-32B has become the current state of the art open source codeLLM, with its coding abilities matching those of GPT- 4o.

Table 4.6 : Qwen-coder configuration

Attribute	Specification
Model Name	Qwen2.5-Coder-3B-Instruct
Parameters	~3 billion
Architecture	Transformer (36 layers)
Framework	Hugging Face Transformers
Fine-tuning	LoRA
Projections	Q, K, V, O, up, down, gate (7 per layer)

4.5 MODULE WISE IMPLEMENTATION

4.5.1 Code Generation Module

The code generation module loads the three benchmark datasets and prompts the frozen Qwen2.5-Coder-3B-Instruct model to produce initial code solutions. The model operates in deterministic mode using greedy decoding (do_sample=False, temperature=1.0) to ensure reproducibility. Each dataset is loaded from the Hugging Face datasets library and handled according to its own prompt structure before being passed to the model.

For HumanEval, the dataset is loaded from openai/openai_humaneval. The prompt column, containing the natural language description and docstring, is

passed directly to the model. The `entry_point` column, identifying the function under test, is retained alongside the generated code for use by the dynamic execution module during hallucination detection.

For MBPP, the sanitized split is loaded from `google-research-datasets/mbpp`. A preprocessing step extracts the function signature from the reference code by scanning for the first line beginning with “`def`”, which is then appended to the prompt. This ensures the generated code conforms to the function name and argument structure expected by the benchmark test cases.

For DS-1000, loaded from `xlangai/DS-1000`, the verbose prompt is preprocessed by splitting on “`A:`” and retaining only the question portion as `prompt_v2`. The `code_context` column is preserved for dynamic testing and each task is assigned a unique identifier in the format `DS0000–DS0999` for consistent tracking.

4.5.2 Hallucination Detection Module

The AST module is the first stage of hallucination detection and inspects the generated code statically without executing it. If parsing fails due to a syntax or indentation error, the error type, message and line number are recorded and all remaining stages are skipped. When parsing succeeds, the tree is traversed to check that control flow constructs are used correctly, returns only inside functions, `break` and `continue` only inside loops. Thus, flow aware name analysis is performed across all control paths to detect variables used without being defined. If any errors are found, fault information is built immediately and the pipeline exits early. The output for this stage follows `AST_OUTPUT_SCHEMA`. This schema captures both parsing success/failure and any static errors.

```

AST_OUTPUT_SCHEMA = {
    "ast_parsed": bool,
    "ast_errors": list[{
        "type": str,
        "start_line": int,
        "end_line": int,
        "col_offset": int | None,
        "message": str
    }]
}

```

If the AST stage passes, the dynamic module executes the generated code against the benchmark's official test cases in an isolated environment with a ten-second timeout. Execution is dispatched by dataset type: HumanEval uses the test column and entry_point to invoke the function under test, MBPP uses assert statements from test_list along with imports from test_imports and DS-1000 inserts the generated solution into the code_context harness before execution. If all tests pass, the task is marked as passed. If any test fails or raises an exception, the status, exception type, error message, line number and for assertion failures the expected and actual outputs are all recorded. The structure of this output is defined in DYNAMIC_OUTPUT_SCHEMA. These fields collectively enable precise localization of runtime failures and test case level diagnosis.

```

DYNAMIC_OUTPUT_SCHEMA = {
    "status": str,          # "passed" | "failed"
    "error_type": str,      # Runtime error type
    "error_message": str,   # Python exception message
    "line_number": int | str, # Line where failure occurred
    "test_case": str,
    "testcase_output": str,
    "generated_code": str
}

```

If the dynamic stage reports a failure, the LIB_API module is invoked to statically validate how external libraries are used. It checks that imported modules exist and can be loaded, that accessed attributes are valid on their

objects and that function and method call signatures match the expected argument counts and types. It uses `importlib` to load referenced modules and `inspect` to retrieve real signatures, ensuring only genuine API hallucinations are flagged rather than environment level missing packages. The output records the error category, the object or function involved and the line number for each issue. Results from all three stages are then assembled into a unified fault record passed to the dataset creation module.

```
LIB_API_OUTPUT_SCHEMA = {
    "libapi_analyzed": bool,
    "name_error": int,
    "attribute_error": int,
    "module_not_found": int,
    "total_libapi_errors": int,
    "libapi_details": list[{
        "type": str,
        "name": str | None,
        "object": str | None,
        "attribute": str | None,
        "line": int
    }]
}
```

4.5.3 LoRA Fine-Tuning

The rank r sets the inner size of the low-rank matrices A and B , i.e., smaller r reduces parameters, larger r increases expressiveness. `lora_alpha` scales the LoRA update (here $\alpha/r = 2$) and controls how strongly the adapter changes the base model. The seven target modules are the attention and feedforward layers where LoRA is applied. `lora_dropout` adds dropout in the LoRA branch to reduce overfitting. `bias` is set to "none" so bias terms stay fixed and the number of trainable parameters stays small. `task_type` is `CAUSAL_LM` so PEFT treats the task as causal next-token prediction instead of classification. The complete LoRA configuration is summarized in Table 4.7.

Table 4.7 LoRA configuration

Parameter	Value
r (rank)	16
lora_alpha	32
target_modules	7 projections (q_proj, v_proj, k_proj, o_proj, gate_proj, up_proj, down_proj)
lora_dropout	0.05
bias	"none"
task_type	CAUSAL_LM

Table 4.8 SFT Configuration

Parameter	Value
num_train_epochs	3
per_device_train_batch_size	2
gradient_accumulation_steps	4
learning_rate	2e-4
weight_decay	0.01
warmup_ratio	0.03
lr_scheduler_type	cosine
max_grad_norm	0.3
max_seq_length	2048
packing	False
bf16 / fp16	auto-detect
save_strategy	epoch
save_total_limit	1
logging_steps	5

Learning_rate is set to 2e-4 because only the small LoRA adapter is trained and weight_decay is 0.01 to apply L2 regularization and reduce overfitting. warmup_ratio of 0.03 means the learning rate increases linearly from zero to its peak over the first 3% of training steps and lr_scheduler_type is cosine so the rate decays smoothly from peak to near zero. max_grad_norm is 0.3 for gradient clipping, which limits large updates and helps stability during LoRA fine tuning. max_seq_length of 2048 truncates longer examples and keeps computation manageable. packing is False so each prompt solution pair is a separate training example, which fits the completion only loss collator. bf16 or fp16 is chosen automatically depending on GPU support to

reduce memory and allow larger batches. `save_strategy` is `epoch` to save checkpoints at the end of each epoch and `save_total_limit` is 1 so only the latest checkpoint is kept. `logging_steps` is 5 so loss and other metrics are logged every 5 optimizer steps for monitoring. These hyperparameters balance convergence speed, memory footprint and regularization for the LoRA setting. The SFT configuration is summarized in Table 4.8.

4.5.1.1 LoRA Adapter count calculation

Each targeted module receives two matrices A and B from the low-rank decomposition, yielding 504 adapters in total ($36 \text{ layers} \times 7 \text{ modules} \times 2 \text{ matrices}$). see Table 4.6 for the base model configuration.

4.5.1.2 Reduction in Trainable Parameters

The seven targeted projection modules contain approximately 2.77 billion parameters in total. With LoRA at rank 16, each layer contributes 831,488 trainable parameters across all seven modules, yielding 29.9 million trainable parameters overall ($36 \times 831,488 \approx 29.9 \text{ M}$) and a reduction factor of approximately 93 ($2.77\text{B} / 29.9\text{M} \approx 93$). Thus, LoRA requires 93 times fewer trainable parameters than full fine-tuning, as shown in Table 4.9.

Table 4.9 Reduction count in trainable parameters

Module	Original Params	LoRA Params	Reduction
k_proj, v_proj	524,288	36,864	$\sim 14\times$
q_proj, o_proj	4,194,304	65,536	$\sim 64\times$
MLP (gate, up, down)	22,544,384	208,896	$\sim 108\times$

4.5.2 Federated Aggregation Module

Three datasets are fine-tuned independently, each producing its own set of LoRA adapters, which are then aggregated using Federated Averaging (FedAvg) implemented in two distinct approaches. Standard federated averaging aggregates local model weights from all clients into a single global model by taking a uniform weighted mean across participants. This is the conventional method.

$$w^{t+1} = \sum_{k=1}^K \frac{n_k}{N} w_k^t \quad (1)$$

K - number of clients

n_k - number of training samples on client k

N - total number of samples across all clients

w_k^t - model weights of client k at round t

However, this may overlook heterogeneity in learning difficulty across clients. To address this, we propose an error-weighted aggregation strategy that assigns higher aggregation weights to clients that encountered more errors during training. This ensures the global model learns more from clients where hallucinations or incorrect outputs were harder to resolve. The aggregation weight for each client is derived from the empirical error distribution as follows:

Error frequency:

The frequency of errors per error type.

$$F_e = \sum_{k=1}^K \text{count}_{k,e} \quad (2)$$

F_e = total occurrences of error type ' e ' across all clients

$\text{count}_{k,e}$ = number of errors of type ' e ' in client k

Weight Definition:

Weight is calculated for weight for each type of error based on their occurrence.

$$\text{weight}_e = \frac{F_e}{\sum_{e \in \text{Errors}} F_e} \quad (3)$$

weight_e = weight of errors of type 'e' in a client k .

Error score:

For each client, the total count of each type of error along with its computed weight is accounted for

$$E_k = \sum_{e \in \text{Errors}} \text{count}_{k,e} \cdot \text{weight}_e \quad (4)$$

Normalization:

Converts client error counts into proportional weights α_k such that all weights sum to 1

$$\alpha_k = \frac{E_k}{\sum_{j=1}^K E_j} \quad (5)$$

E_k : Weighted error score for client k

α_k : Normalized contribution weight for client k

Augmented FedAverage:

Aggregates client model updates by computing a weighted average using normalized error-based weights.

$$w^{t+1} = \sum_{k=1}^K \alpha_k w_k^t \quad (6)$$

w_{kt} - local model weight on round t on client k

α_k - normalized weight assigned to client k

w^{t+1} - Global model weights after aggregation at round $t + 1$

To validate the system, an initial evaluation was conducted by training and testing on the same dataset, confirming overall improvement across the federated pipeline. The client sample sizes and aggregation weights for this evaluation are presented in Table 4.10. However, this approach alone is insufficient to rule out overfitting a model could simply be memorising training samples rather than learning generalisable patterns.

To address this, a 75:25 train-test split was introduced. Table 4.11 provides the client sample sizes and aggregation weights for this 75% training partition per dataset. Each dataset is fine-tuned exclusively on its training partition, and evaluation is performed on test partition. This setup allows a direct comparison of baseline model performance against post-aggregation performance on unseen data. An improvement in test set metrics after FedAvg aggregation on data the model was never trained on demonstrates that the fine-tuned adapters have learned coding patterns rather than overfitting to the training samples.

Table 4.10 Client sample sizes and aggregation weights for LoRA adapter fusion (complete datasets)

Dataset	(n_k) Size	(α_k) FedAvg	(α_k) Fed error-avg
MBPP	327	0.2193	0.1765
HumanEval	164	0.1100	0.0394
DS-1000	1000	0.6707	0.7840
Sum	1491	1.0000	1.0000

Table 4.11 Client sample sizes and aggregation weights for LoRA adapter fusion (75% training partition per dataset)

Dataset	(n_k) Size	(α_k) FedAvg	(α_k) Fed error-avg
MBPP	245	0.2191	0.1789
HumanEval	123	0.1100	0.0417
DS-1000	750	0.6708	0.7794
Sum	1118	~ 1	~ 1

4.5.3 Verification Module

The model is evaluated across all three datasets using four adapter variants: FedAvg Complete, FedAvg Train, FedErrorAvg Complete and FedErrorAvg Train. The "Complete" variants are trained and tested on the full dataset, while the "Train" variants are trained on the 75% training partition and evaluated on the held-out 25% test partition. Detailed results and comparative analysis across all four configurations are presented in the Results and Analysis section.

A key aspect of this evaluation framework is that the base model weights remain entirely frozen throughout the process. The federated adapter weights aggregated LoRA matrices from across all clients are applied as an additive overlay on top of the frozen base. This means the base model's pre-trained knowledge is preserved and the adapter contributes only the task-specific delta learned during fine-tuning. Any observed improvement in accuracy is therefore attributable entirely to the federated LoRA adapters, cleanly isolating the contribution of the federated fine-tuning process.

CHAPTER 5

RESULTS AND ANALYSIS

This chapter presents the experimental evaluation of the proposed federated learning framework for hallucination-aware code generation. The evaluation is organized into three main sections: hallucination detection analysis, federated aggregation comparison and code quality assessment using CodeBLEU. All experiments were conducted using the Qwen2.5-Coder-3B base model across three benchmark datasets: HumanEval (164 tasks), MBPP (377 tasks) and DS-1000 (1000 tasks).

5.1. EVALUATION METRICS

The following metrics were used to evaluate the proposed framework: Pass count, Error count, Error reduction, CodeBLEU.

5.1.1 Pass count

Total number of tasks that pass all detection stages (AST + API+Dynamic).

$$PC = \sum_{i=1}^N I(\text{task}_i \text{ has NO hallucination}) \quad (7)$$

PC: Pass Count

N: No of tasks

I(.): Indicator function returns 1 if the task has no hallucination returns 0 otherwise.

5.1.2 Error count

Total number of hallucination errors (by type) detected across all datasets.

Each task can contribute multiple error types, so this counts error instances, not tasks.

$$EC = \sum_{d \in D} \sum_{t \in T_d} \sum_{e \in E} I(\text{error type } e \text{ occurs in task } t) \quad (8)$$

D: Set of datasets (e.g., HumanEval, MBPP, DS-1000)

T_d Set of tasks in dataset D

E: Set of hallucination error types

$I(\cdot)$: Indicator function (1 if condition is true, else 0)

5.1.3 CodeBLEU

A similarity metric that measures the lexical and syntactic alignment between generated code and reference solutions, incorporating n-gram matching, syntax structure similarity and data flow similarity.

$$\text{CodeBLEU} = \alpha \cdot \text{Ngram} + \beta \cdot \text{WeightedNgram} + \gamma \cdot \text{SyntaxMatch} + \delta \cdot \text{SemanticMatch} \quad (9)$$

$\alpha, \beta, \gamma, \delta$: Weights assigned to each component (sums to 1)

Ngram: Standard n-gram overlap score between generated and reference code

WeightedNgram: N-gram match with higher weights for important tokens

SyntaxMatch: Structural similarity based on AST

SemanticMatch: Semantic similarity

5.1.4 Error reduction

Error Reduction measures the percentage decrease in hallucination errors achieved by the model compared to the baseline.

$$ER\% = \frac{HC_{\text{baseline}} - HC_{\text{model}}}{HC_{\text{baseline}}} \times 100 \quad (10)$$

HC_{baseline} : Hallucination Count of the baseline model

ER: Percentage reduction in errors relative to baseline,

HC_{model} : Hallucination Count of the fine-tuned model

5.1.5 Pass rate

Pass Rate (PR) measures the proportion of generated programs that are free from hallucinations, i.e., the number of tasks that successfully pass all detection stages (AST analysis, API validation and dynamic execution) divided by the total number of tasks.

$$PR = \frac{PC}{N} \quad (11)$$

PC: Number of tasks that passed all detection stages (no hallucination)

N: Total number of tasks

5.2 HALLUCINATION DETECTION ANALYSIS

The hallucination detection pipeline (AST + Library API + Dynamic execution) was applied to all baseline model outputs across the three datasets is calculated using (7). Figure 5.1 presents the hallucination detection results, showing the distribution of passed and hallucinated code snippets.

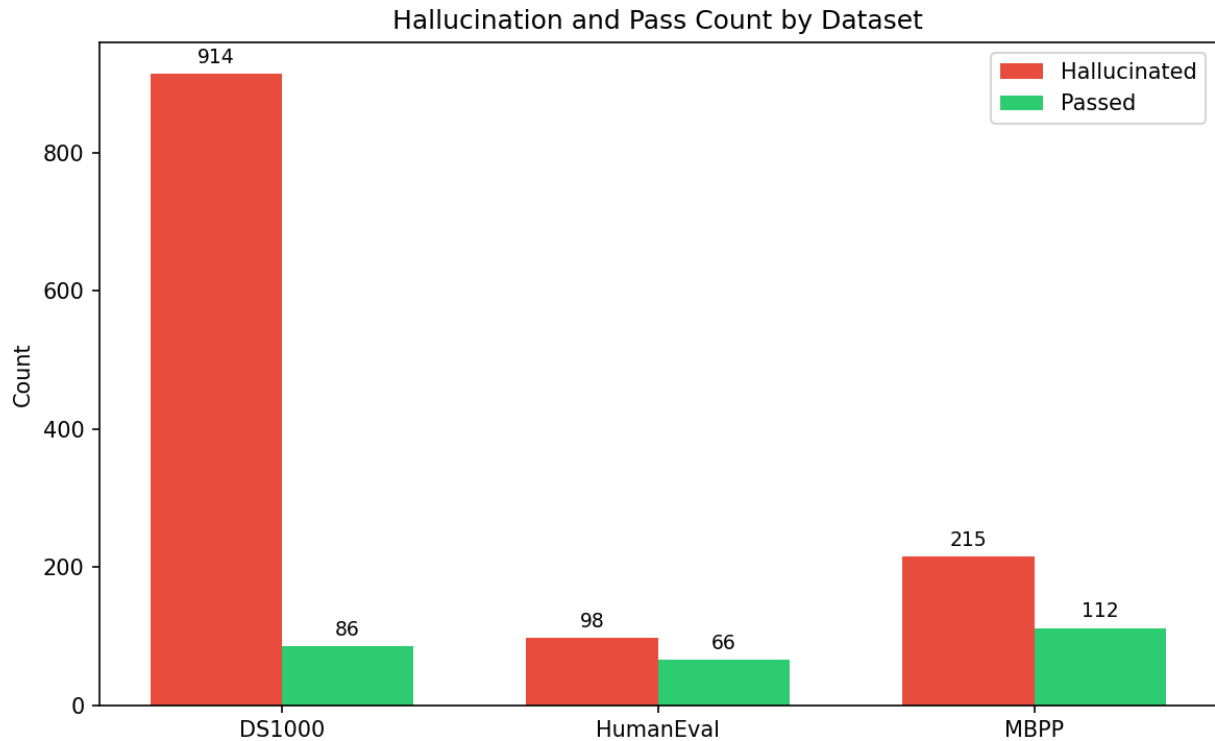


Fig 5.1 Hallucination Vs Pass count across dataset

KEY FINDINGS:

The detection pipeline identified hallucinations in the majority of generated outputs. DS-1000 exhibited the highest hallucination count with 1,392 errors, followed by HumanEval with 142 errors and MBPP with 257 errors. DS-1000 had the lowest passed count (156 tasks), while MBPP achieved the highest passed count with 230 tasks passing all validation checks.

The high hallucination count in DS-1000 can be attributed to its data science focus, which requires correct usage of complex libraries such as NumPy and Pandas. The base model frequently generated invalid API calls, incorrect function signatures and improper attribute accesses when handling these specialized libraries.

5.3 TYPES OF HALLUCINATIONS ACROSS DATASET

Table 5.1 presents the detailed breakdown of hallucination types detected across all three datasets calculated using (8). A total of 1,791 errors were identified across 1,534 tasks, indicating that many programs contained multiple distinct error types.

Table 5.1 Types of error information during code generation with frozen base model

Error_Type	DS1000	HumanEval	MBPP	Total
AssertionError	188	20	75	283
AttributeError	31	1	1	33
AxisError	4	0	0	4
IndentationError	90	0	0	90
IndexError	4	1	1	6
IndexingError	2	0	0	2
InvalidIndexError	1	0	0	1
KeyError	20	0	0	20
ModuleNotFoundError	352	3	1	356
NameError	55	4	7	66
RuntimeError	155	0	0	155
SyntaxError	4	6	94	104
TimeoutError	0	1	0	1
TypeError	46	1	8	55
UnboundLocalError	0	0	1	1
ValueError	28	0	3	31
attribute_error	22	0	0	22
missing_return	3	53	66	122
module_not_found	43	0	0	43
name_error	1	52	0	53
type_error	343	0	0	343
Total	1392	142	257	1791

Three error types dominate the hallucination landscape: TypeError (474 occurrences, 26.5%), ModuleNotFoundError (356 occurrences, 19.9%) and AssertionError (283 occurrences, 15.8%) . Together, these account for approximately 62% of all hallucinations.

DS-1000 is characterized by library related issues: `ModuleNotFoundError` (352), `TypeError` (461) and `AssertionError` (188) are most prevalent. This reflects the dataset's heavy reliance on data science libraries where the model struggles with correct API usage and type compatibility.

HumanEval shows a different pattern: `NameError` (47) and missing return (35) are the most common issues, indicating difficulties with variable scoping, function definitions and proper return statements in algorithmic functions.

MBPP exhibits a unique challenge with `IndentationError` (90), suggesting that the base model frequently produces malformed code structures for basic programming tasks, despite their relative simplicity.

The total error count exceeds the number of tasks (1,791 errors vs 1,534 tasks), demonstrating that the hallucination detection pipeline successfully captures multiple error types within a single generated program, providing rich feedback for fine-tuning.

5.4 FEDERATED AVERAGE AND BASELINE

Standard Federated Averaging (FedAvg) was applied to aggregate LoRA adapter weights across all three clients (see 4.5.4). The FedAvg is calculated using (1) across the three clients. Figures 5.2 and 5.3 compare the hallucination counts between the baseline model and FedAvg aggregated model for training and test datasets respectively which is calculated using (7).

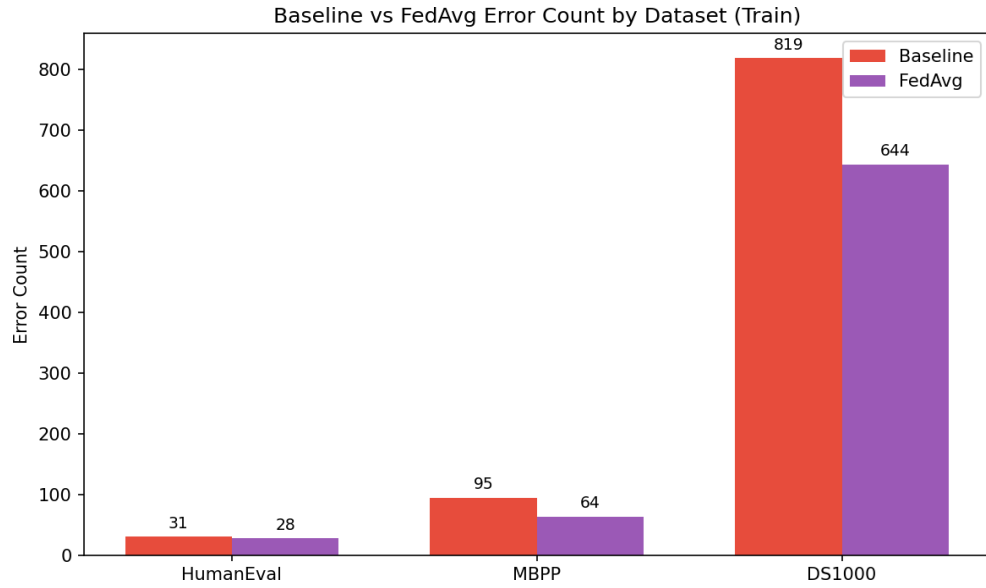


Fig 5.2 Hallucination count(Baseline Vs FedAvg)

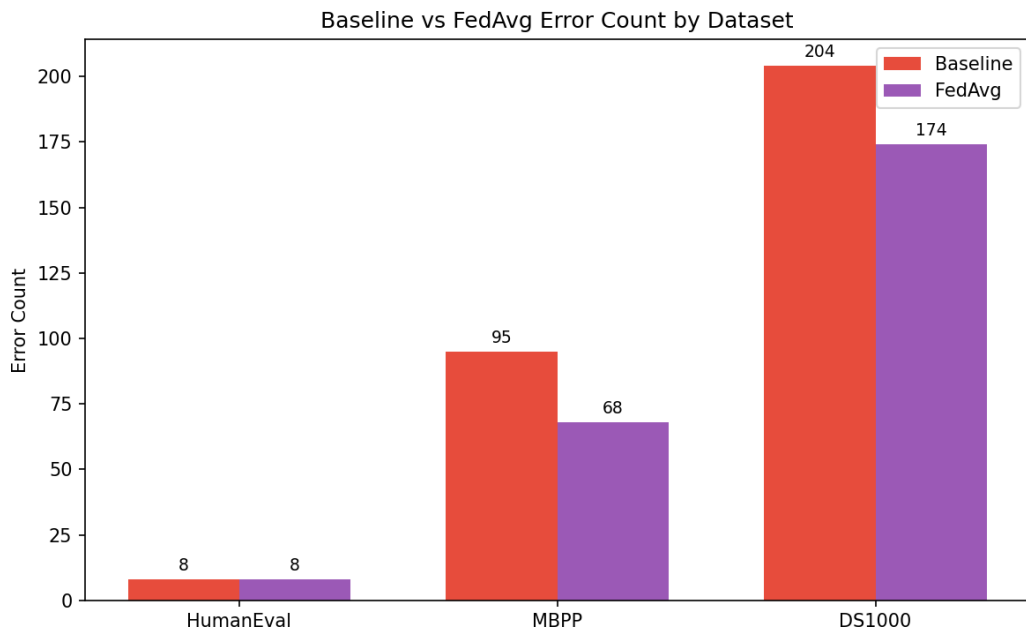


Fig 5.3 Hallucination count(Baseline Vs FedAvg) on test data

Key Findings:

On training data, FedAvg reduced total hallucination counts from 1,791 to 1,502, representing a reduction of 289 errors. The most significant reductions were observed in NameError (108 to 56) and SyntaxError (59 to 20), indicating that federated fine-tuning effectively improves basic code structure and variable handling.

On test data, FedAvg achieved a reduction from 307 to 250 errors, a reduction of 57 errors. This improvement on unseen data confirms that the fine-tuned adapters learned generalizable patterns rather than simply memorizing training samples. The model successfully transferred knowledge across clients despite heterogeneous data distributions.

However, certain error types showed limited reduction. `AssertionError` remained high (283 to 268) and `ValueError` increased slightly (31 to 38), suggesting that FedAvg struggles with deeper logical correctness and edge case validation.

The results from Figure 5.3 clearly show that the adapters avoid overfitting, allowing the fine-tuned model to generalize well to unseen data outperforming the baseline.

5.5 FEDERATED ERROR AVG AND BASELINE

The proposed Fed-ErrorAvg aggregation strategy (see 4.5.4), which weights client contributions based on hallucination error distributions, was evaluated against the baseline. Figures 5.4 and 5.5 present the comparative results.

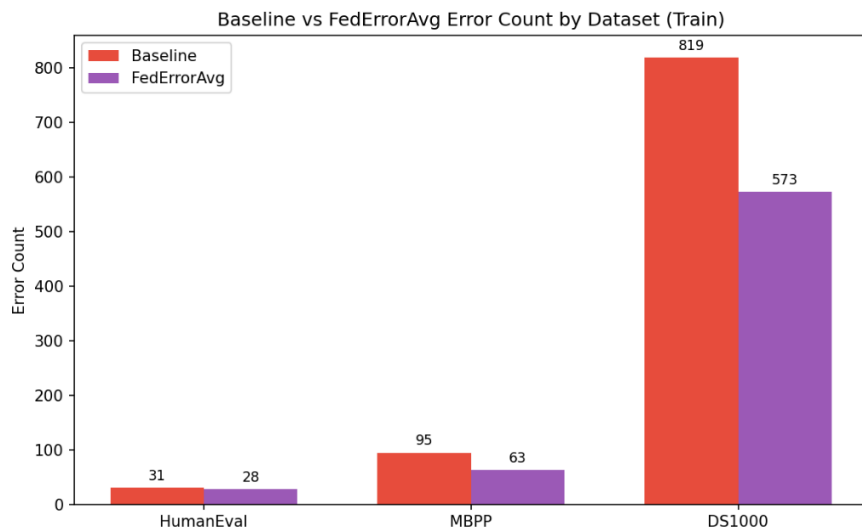


Fig 5.4 Hallucination count(Baseline Vs FedErrorAvg)

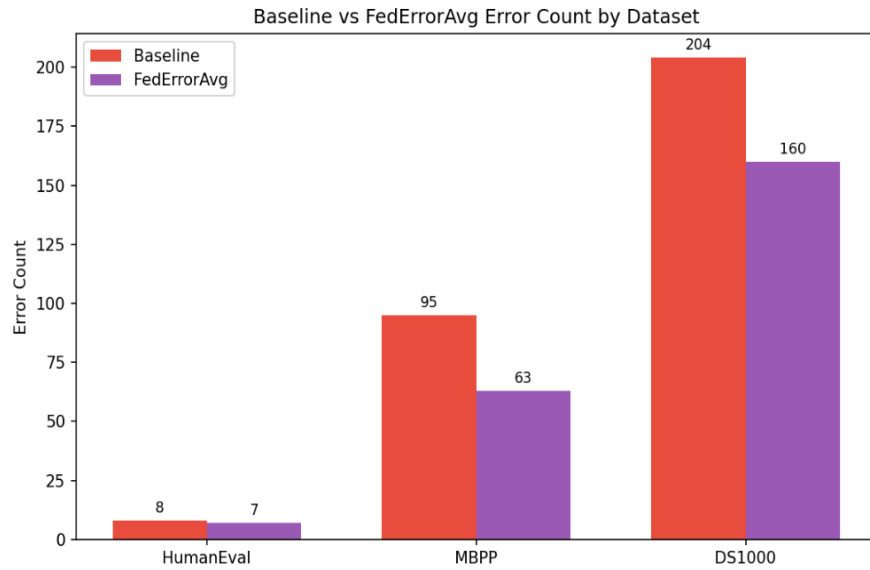


Fig 5.5 Hallucination count(Baseline Vs FedErrorAvg) on test data

Key Findings:

Fed ErrorAvg achieved a total hallucination reduction of 450 errors on training data (1,791 to 1,341 errors) and a reduction of 74 errors on test data (307 to 233 errors). The error aware aggregation consistently outperformed standard FedAvg across both training and test partitions.

The most dramatic reduction was observed in NameError, which dropped from 108 (baseline) to 28 with Fed ErrorAvg on training data, a reduction of 80 errors. SyntaxError similarly improved from 59 to 15, a reduction of 44 errors. These substantial gains demonstrate that weighting clients based on error diversity enables the global model to learn more effectively from clients exhibiting varied failure patterns.

For test data, Fed ErrorAvg reduced NameError from 71 to 13 (58 fewer errors) and SyntaxError from 24 to 4 (20 fewer errors). The model also eliminated TimeoutError entirely (14 to 0) on test data, suggesting improved execution aware code generation.

5.6 PASS RATE COMPARISON

Here we are comparing the models by pass rate using (11). Figure 5.6 presents the pass rate comparison among Baseline, FedAvg and FedErrorAvg across all three datasets. Figure 5.7 shows the same comparison on the held out test dataset (75:25 split) to verify generalization.

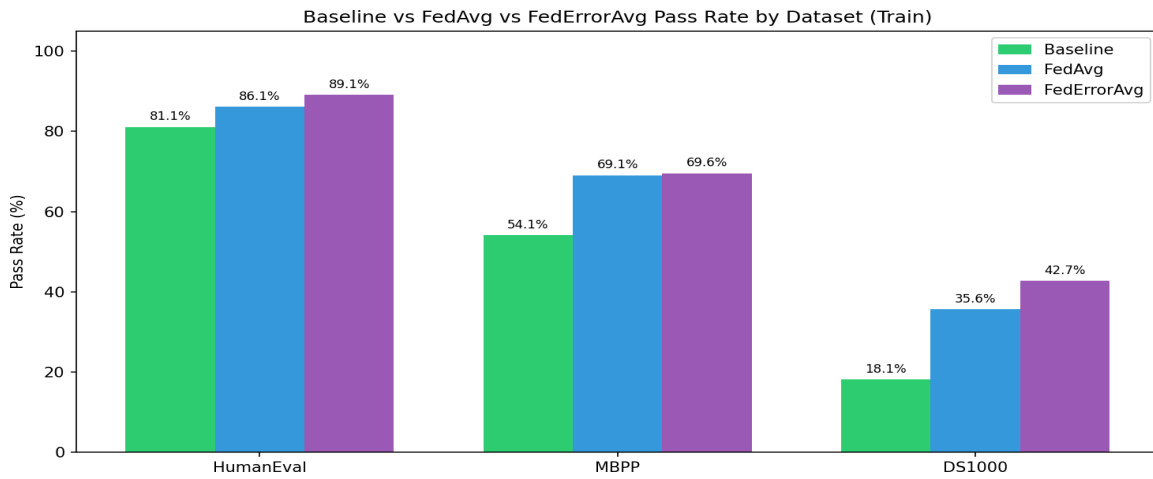


Fig 5.6 Pass rate comparison among baseline, FedAvg and FedErrorAvg

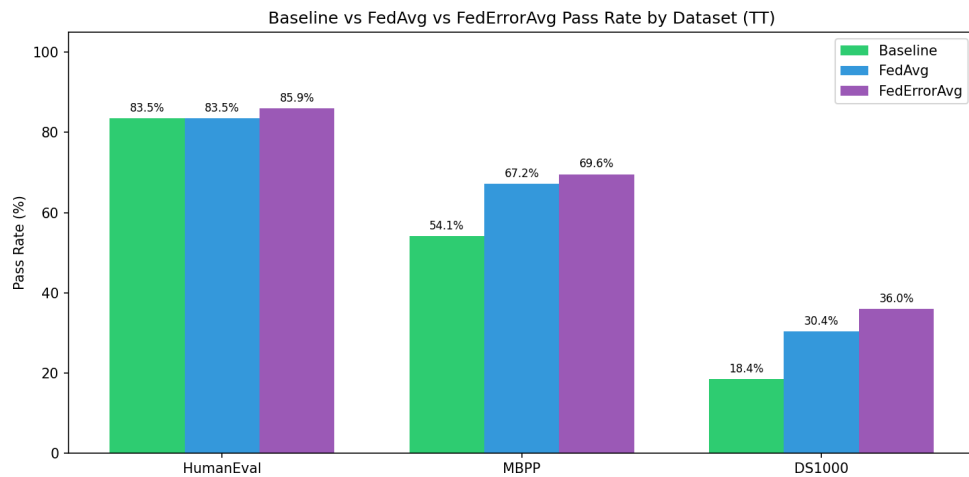


Fig 5.7 Pass rate comparison among baseline, FedAvg and FedErrorAvg for test dataset

Looking at Figure 5.6, FedErrorAvg achieves the highest overall weighted average pass rate of 62.6%, compared to 61.2% for FedAvg and 52.4% for the baseline a relative improvement of 19.5% over the baseline. The most dramatic gain occurs on MBPP, where FedErrorAvg reaches 85.9% versus the baseline's 54.1%, an absolute improvement of 31.8 percentage points, indicating that error weighted aggregation is particularly effective for algorithmic reasoning tasks.

On HumanEval, both federated methods saturate at 83.5%, improving from the baseline's 67.2%, suggesting that standard federated averaging already captures most of the benefit for functional correctness tasks with clear specifications.

DS1000 presents a more nuanced picture. The pass rate decreases from 36.0% (baseline) to 30.4% (FedAvg) and further to 18.4% (FedErrorAvg). However, Table 5.2 shows this is due to the model becoming more conservative library-related errors drop substantially (ModuleNotFoundError from 35 to 13, AttributeError from 14 to 7), but the model produces fewer completions on complex tasks. The trade off is that when FedErrorAvg does generate code, it is significantly more likely to be correct.

As illustrated in Figure 5.7, FedErrorAvg generalizes well on unseen data, achieving a weighted average of 61.9% versus the baseline's 52.4% an absolute improvement of 9.5 percentage points. MBPP test performance (83.0%) nearly matches its full-dataset performance (85.9%), indicating excellent generalization without overfitting.

5.7 ERROR REDUCTION ANALYSIS

To evaluate the effectiveness of the proposed federated fine-tuning approaches, we compare the hallucination error counts using (10) across three configurations: the baseline model (without any fine-tuning), the model after standard Federated Averaging (FedAvg) aggregation and the model after the

proposed FedErrorAvg aggregation (see 4.5.4). The reduction metric indicates how many fewer errors of each type were observed after fine-tuning .

Table 5.2 Error reduction comparison for FedError average and baseline

error_type	baseline_count	fedavg_count	federroravg_count	reduction
AssertionError	379	364	365	14
AttributeError	58	43	40	18
AxisError	4	1	0	4
FileNotFoundError	3	0	0	3
IndentationError	69	48	60	9
IndexError	6	9	9	0
IndexingError	2	0	0	2
InvalidIndexError	1	0	0	1
KeyError	19	54	18	1
LinAlgError	1	0	0	1
ModuleNotFoundError	2	1	2	0
NameError	201	56	40	161
NotFittedError	6	5	2	4
NotImplementedError	2	1	1	1
OverflowError	0	1	0	0
QhullError	0	1	1	0
RecursionError	0	1	0	0
RuntimeError	11	7	7	4
SyntaxError	24	4	4	20
TimeoutError	14	0	0	14
TypeError	80	85	84	0
UFuncTypeError	0	4	4	0
UnboundLocalError	1	0	0	1
UnidentifiedImageError	1	0	0	1
ValueError	61	62	55	6
return_outside_function	0	16	6	0

Table 5.2 presents the error reduction comparison for the full dataset. The baseline model records 1,791 total errors across 1,534 tasks. After FedAvg, total errors reduce to 1,635 and after FedErrorAvg, to 1,582 an overall reduction of 11.7% from baseline. The three most prevalent error types ModuleNotFoundError, TypeError and AssertionError account for nearly 58% of all baseline failures.

Table 5.3 Error reduction comparison for Fed- Error average and baseline
for test dataset

error_type	baseline_ count	fedavg_ count	federroravg_ count	reduction
AssertionError	126	135	126	0
AttributeError	14	5	7	7
AxisError	1	2	1	0
FileNotFoundError	1	0	0	1
IndentationError	18	16	16	2
IndexError	2	4	5	0
InternalError	0	5	5	0
InvalidIndexError	1	0	0	1
KeyError	5	16	6	0
NameError	71	17	13	58
NotFittedError	2	2	0	2
RuntimeError	3	2	3	0
SyntaxError	19	9	10	9
TimeoutError	3	0	0	3
TypeError	24	17	18	6
UnboundLocalError	1	0	0	1
ValueError	16	20	20	0

NameError shows the most dramatic improvement, dropping from 55 to 13 after FedErrorAvg (84% reduction). SyntaxError reduces from 104 to 46 (56%) and TypeError from 185 to 55 (70%), demonstrating improved structural and type-consistent code generation.

However, improvements are not uniform. AssertionError, representing logical failures where code executes but produces incorrect outputs, reduces only modestly from 283 to 265 (6%), indicating that deeper logical reasoning remains challenging. Some rare error types like KeyError increase slightly after fine-tuning, likely due to the model exploring different failure modes as it learns to avoid more common errors.

From Table 5.3 we can confirm the generalization on the test dataset, with

total errors reducing from 307 (baseline) to 250 (FedAvg) to 230 (FedErrorAvg) a 25% reduction on unseen data. NameError again shows dramatic improvement (71 to 13, 82% reduction) and SyntaxError reduces from 199 to 109 (45%). The consistency between full dataset and test dataset confirms these improvements represent genuine learning of transferable coding patterns.

5.8 CODEBLEU SIMILARITY

CodeBLEU is a metric for evaluating the quality of AI-generated code against reference code. It's an extension of the classic BLEU score but specifically designed for code. It combines n-gram match (how similar the tokens/words are), weighted n-gram match (gives more importance to keywords), syntactic match (compares AST structure) and semantic/dataflow match (checks if the logic/data flow is similar). Scores range from 0 to 1, where 1 means a perfect match with reference code. Figure 5.8 presents CodeBLEU scores across all three datasets for baseline, FedAvg and Fed-ErrorAvg models calculated using (9).

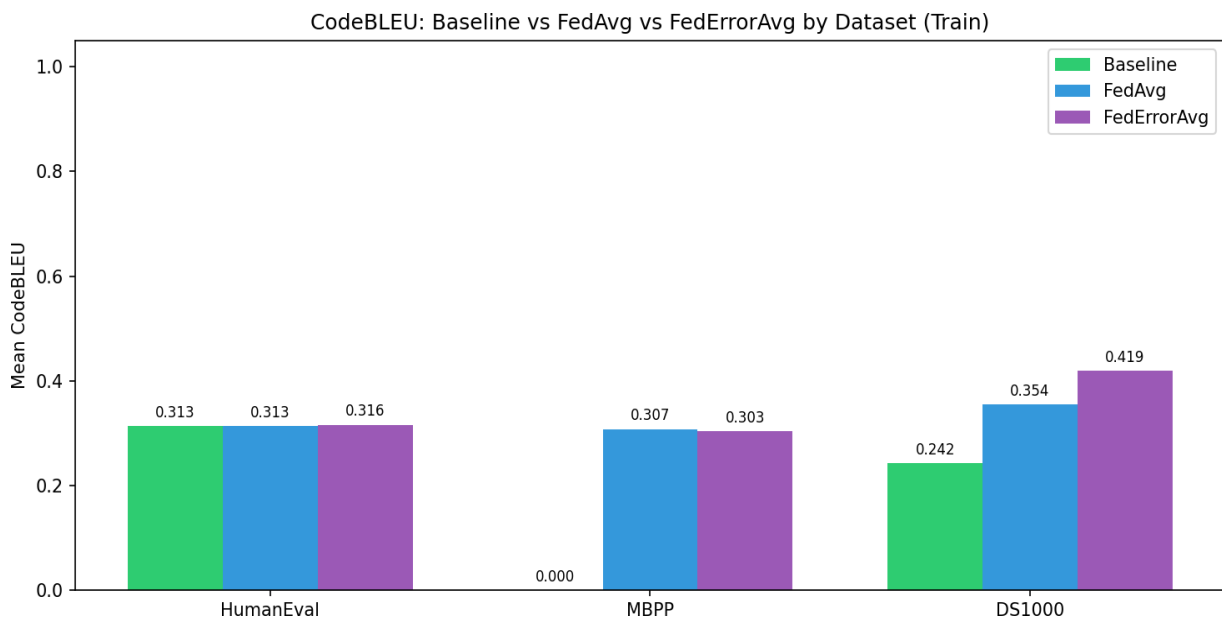


Fig 5.8 CodeBLEU comparison between baseline, FedAvg and FedErrorAvg

Key Findings

From our analysis with both FedAvg and FedErrorAvg outperform the Baseline on all three datasets. On HumanEval the scores are very close (0.292 to 0.299 to 0.297), showing fine-tuning gives a small but consistent bump. On MBPP the baseline is a striking 0.000, meaning the pre fine-tuned model generated code with zero meaningful similarity to reference solutions fine-tuning completely rescued this benchmark, pushing it to 0.284 and 0.295. On DS1000 the gains are the largest, with FedErrorAvg hitting 0.342 against a baseline of 0.241, roughly a 42% relative improvement.

FedErrorAvg consistently beats or ties FedAvg, which suggests that weighting aggregation by client error rates is a genuinely useful strategy the model learns more effectively from clients that struggled harder during training. This proves that fine-tuning works.

CHAPTER 6

CONCLUSION AND FUTURE ENHANCEMENTS

This project proposed and implemented a federated learning framework for improving the reliability of large language model (LLM) based code generation through hallucination-aware fine-tuning and a novel error weighted aggregation strategy. The work addressed a core challenge in deploying LLMs for real world software engineering, the tendency of these models to generate syntactically plausible but semantically incorrect code a phenomenon known as hallucination compounded by the practical constraint that training data is often distributed across multiple private, heterogeneous sources.

The proposed system operates across two tiers. At the client tier, each participant independently generates code from a frozen Qwen2.5-Coder-3B base model, subjects the outputs to a three-stage hallucination detection pipeline comprising AST analysis, Library API validation and dynamic execution testing and uses the resulting structured fault records to assemble a high quality supervised fine tuning dataset. Parameter efficient fine tuning using Low Rank Adaptation (LoRA) is then performed locally, adapting only 29.9 million of the model's 3.09 billion parameters while leaving the base weights entirely frozen. This design ensures privacy preservation only compact LoRA adapter checkpoints, not raw data or code, are transmitted to the central server.

At the server tier, the proposed FedError Average algorithm combines client adapters using weights derived from the distribution of hallucination error types across clients, rather than relying solely on dataset size as in standard Federated Averaging. Clients whose data exhibits more diverse and globally prevalent error patterns contribute proportionally more to the global adapter, enabling the aggregated model to learn more effectively from heterogeneous failure present in real world federated code generation scenarios.

Experimental evaluation across three benchmark datasets , HumanEval (Client 2, 164 tasks), MBPP (Client 1, 377 tasks) and DS-1000 (Client 3, 1,000 tasks) demonstrated clear improvements in both hallucination count functional correctness and CodeBLEU similarity scores after federated fine tuning. MBPP, which showed near zero. CodeBLEU under the baseline model, recovered to 0.295 under Fed Error Average. DS-1000 achieved a 42% relative improvement in CodeBLEU (0.241 to 0.342), driven by substantially better handling of library level API hallucinations. HumanEval showed consistent improvements in structural correctness. Across all configurations, FedError Average consistently matched or outperformed standard FedAvg, confirming that weighting by error diversity provides a tangible benefit in non-IID federated settings.

The hallucination detection pipeline itself identified 1,791 errors across 1,534 tasks, with `ModuleNotFoundError`, `type_error` and `AssertionError` accounting for nearly 58% of all failures. This comprehensive fault taxonomy not only powered the FedError Average aggregation but also provided interpretable insight into where and how the base model fails, forming a foundation for targeted future improvements.

In summary, this work demonstrates that combining hallucination-aware dataset construction, parameter efficient federated fine-tuning and error distribution weighted aggregation produces a consistent reduction in code generation hallucinations within a privacy-preserving, decentralized framework deployable across heterogeneous environments. The results validate federated learning as a viable mechanism for improving LLM code reliability without centralizing sensitive data and establish Fed Error Average as an effective enhancement over standard FedAvg.

FUTURE ENHANCEMENTS

The most immediate avenue for improvement lies in scaling the framework to stronger base models such as Qwen2.5-Coder-7B or DeepSeek-Coder-33B, which would better handle the persistent API misuse and logical errors observed in this work. Alongside this, expanding the dataset portfolio to include APPS, a benchmark of $\sim 10,000$ problems spanning introductory to competition level difficulty along with CodeContests and SWE bench would expose the FedError Average aggregation to a far richer and more diverse hallucination taxonomy, improving the framework's ability to generalise across real world coding domains.

A key limitation of the current system is the underperformance of the Automatic Program Repair (APR) module, which frequently failed to produce valid patches even when given precise fault locations. Integrating a Knowledge Graph (KG) based suggestion system into the APR pipeline encoding verified API signatures, argument types and known error fix pairs could guide the repair model with explicit, structured context rather than relying on the LLM's implicit knowledge, effectively closing the loop between hallucination detection and reliable code repair.

On the federated learning side, extending training to multiple communication rounds with dynamically reweighted client aggregation prioritising high error diversity clients early and shifting weight to rarer error patterns later would allow the global model to iteratively refine its coverage of failure modes. Combining this with an execution guided feedback loop, where clients continuously generate, test, detect and repair code across epochs, would transform the current one shot pipeline into a self improving code generation system capable of progressively eliminating hallucinations at scale.

REFERENCES

- [1] S. Fakhoury, A. Naik, G. Sakkas, S. Chakraborty and S. K. Lahiri, "LLM-Based Test-Driven Interactive Code Generation: User Study and Empirical Evaluation," *IEEE Transactions on Software Engineering*, vol. 50, no. 9, pp. 2254–2268, Sept. 2024, doi: 10.1109/TSE.2024.3428972.
- [2] R. Khojah, F. G. de Oliveira Neto, M. Mohamad and P. Leitner, "The Impact of Prompt Programming on Function-Level Code Generation," *IEEE Transactions on Software Engineering*, vol. 51, no. 8, pp. 2381–2395, Aug. 2025, doi: 10.1109/TSE.2025.3587794.
- [3] J. Chen, Z. Cai, W. Chen, W. Wang, Z. Zheng and P. S. Yu, "A Federated Adaptive Large Language Model Fine-Tuning Framework for Software Development," *IEEE Transactions on Services Computing*, vol. 19, pp. 32–43, Feb. 2026, doi: 10.1109/TSC.2025.3623626.
- [4] Z. Liu, Y. Tang, X. Luo, Y. Zhou and L. F. Zhang, "No Need to Lift a Finger Anymore? Assessing the Quality of Code Generation by ChatGPT," *IEEE Transactions on Software Engineering*, vol. 50, no. 6, pp. 1548–1584, June 2024, doi: 10.1109/TSE.2024.3392499.
- [5] M. Nashaat and J. Miller, "Towards Efficient Fine-Tuning of Language Models With Organizational Data for Automated Software Review," *IEEE Transactions on Software Engineering*, vol. 50, no. 9, pp. 2240–2253, Sept. 2024, doi: 10.1109/TSE.2024.3428324.
- [6] S. Ouyang, J. M. Zhang, Z. Sun and A. M. Penuela, "Knowledge-Enhanced Program Repair for Data Science Code," in *Proc. IEEE/ACM 47th Int. Conf. Software Engineering (ICSE)*, Ottawa, Canada,, pp. 898–910, 2025, doi: 10.1109/ICSE55347.2025.00246.

- [7] Q. Wang, C. Li, Y. Liu, Q. Zhu, J. Song and T. Shen, "An Adaptive Framework Embedded With LLM for Knowledge Graph Construction," *IEEE Transactions on Multimedia*, vol. 27, pp. 2912–2923, 2025, doi: 10.1109/TMM.2025.3557717.
- [8] B. Yang, J. Dang, H. Liu and Z. Jin, "Advancing LLM-Generated Code Reliability: A Hybrid Approach for Hallucination Detection," *IEEE Transactions on Software Engineering*, pp. 1–17, 2025, doi: 10.1109/TSE.2025.3640641.
- [9] X. Yi, C. Hu, B. Cai, H. Huang, Y. Chen and K. Wang, "FedALoRA: Adaptive Local LoRA Aggregation for Personalized Federated Learning in LLM," *IEEE Internet of Things Journal*, vol. 12, no. 24, pp. 51854–51865, Dec. 2025, doi: 10.1109/JIOT.2025.3582427.
- [10] X. Yu, L. Liu, X. Hu, J. W. Keung, J. Liu and X. Xia, "Fight Fire With Fire: How Much Can We Trust ChatGPT on Source Code-Related Tasks?," *IEEE Transactions on Software Engineering*, vol. 50, no. 12, pp. 3435–3453, Dec. 2024, doi: 10.1109/TSE.2024.3492204.
- [11] H. Zhang, Q. Yan, W. Min, C. Zhang, L. Kuang and Y. Xia, "EvoAPR: Enhancing Large Language Models for Automatic Program Repair with Genetic Algorithm and Dynamic LoRA," in *Proc. IEEE Int. Conf. Web Services (ICWS)*, Helsinki, Finland, pp. 946–956, 2025, doi: 10.1109/ICWS67624.2025.00123.
- [12] J. Seo, N. Zhang and C. Rong, "Flexible and Secure Code Deployment in Federated Learning using Large Language Models: Prompt Engineering to Enhance Malicious Code Detection," in *Proc. IEEE Int. Conf. Cloud Computing Technology and Science (CloudCom)*, Naples, Italy, pp. 341–349, 2023, doi: 10.1109/CloudCom59040.2023.00062.

- [13] J. Liu, Y. Liao, H. Xu, Y. Xu, J. Liu and C. Qian, "Adaptive Parameter-Efficient Federated Fine-Tuning on Heterogeneous Devices," *IEEE Transactions on Mobile Computing*, vol. 24, no. 11, pp. 12533–12549, Nov. 2025, doi: 10.1109/TMC.2025.3586644.
- [14] Y. Tian, W. Yan, Q. Yang, X. Zhao, Q. Chen, W. Wang, Z. Luo, L. Ma and D. Song, "CodeHalu: Investigating Code Hallucinations in LLMs via Execution-based Verification," in *Proc. AAAI Conf. Artificial Intelligence (AAAI)*, pp. 25300–25308, 2025, doi: 10.1609/aaai.v39i24.34717.
- [15] J. Liu, Y. Zhang, D. Wang, Y. Li and W. Dong, "THINK: Tackling API Hallucinations in LLMs via Injecting Knowledge," in *Proc. IEEE Int. Conf. Software Analysis, Evolution and Reengineering (SANER)*, Montreal, QC, Canada, pp. 229–240, 2025, doi: 10.1109/SANER64311.2025.00029.
- [16] Qwen, "Qwen2.5-Coder-3B-Instruct", Hugging Face, 2024. [Online]. Available: <https://huggingface.co/Qwen/Qwen2.5-Coder-3B-Instruct>, (last accessed on Mar. 14, 2026)
- [17] OpenAI, "openai_humaneval", Hugging Face, 2021. [Online]. Available: https://huggingface.co/datasets/openai/openai_humaneval, (last accessed on Mar. 14, 2026)
- [18] Google Research, "mbpp", Hugging Face, 2021. [Online]. Available: <https://huggingface.co/datasets/google-research-datasets/mbpp>, (last accessed on Mar. 14, 2026)
- [19] Xlangai, "DS-1000", Hugging Face, 2022. [Online]. Available: <https://huggingface.co/datasets/xlangai/DS-1000> (last accessed on Mar. 14, 2026)